

Data Structures and Algorithms

Lab 1 - Algorithm Analysis and Complexity

Comparing Different Approaches to the Same Problem

Lab Overview

This first laboratory introduces an important habit in Data Structures and Algorithms: a problem may have more than one correct solution, but those solutions may behave very differently as the input becomes larger.

The goal of this lab is not simply to produce working Java programs. You should learn to analyze an algorithm, estimate its time and auxiliary-space complexity, compare alternative approaches, and explain why one approach may be more suitable than another under particular constraints.

Core Concepts

Algorithm

An algorithm is a finite sequence of clear steps used to solve a problem. A good algorithm must not only produce the correct result; it should also use time and memory appropriately for the intended input size.

Input Size

The symbol n is commonly used to represent the size of the input. For an array problem, n usually means the number of elements in the array.

When analyzing an algorithm, ask:

- How does the amount of work grow when n becomes larger?
- How much additional memory is required as n becomes larger?

Time Complexity

Time complexity describes how the amount of work performed by an algorithm grows with the input size.

Common complexities used in this lab include:

Complexity	Meaning	Typical Example
$O(1)$	Constant amount of work	Accessing one array element by index
$O(n)$	Work grows approximately linearly with input size	Traversing an array once
$O(n^2)$	Work grows approximately with the square of input size	Comparing every pair of array elements

Big-O notation describes growth rather than exact execution time on a particular computer.

Auxiliary Space Complexity

Auxiliary space describes the additional memory used by an algorithm, excluding the input itself.

Examples:

- A few variables such as counters or running values normally require $O(1)$ extra space.
- An additional array, set, or map that may contain up to n values normally requires $O(n)$ extra space.

Why Compare Different Approaches?

Suppose one value is missing from a known range represented in an array.

One approach searches the array separately for each possible value. It uses constant extra memory but may repeat many comparisons.

Another records which values are present in a boolean array, then inspects those marks. It uses additional memory but avoids searching the original array repeatedly.

Neither description is complete by saying only that one solution is “better.” The appropriate choice depends on the problem constraints, memory limits, number of operations, and simplicity required.

General Requirements

Programming Requirements

- Use Java for all implementations.
- Organize source code clearly by problem and approach.
- Use meaningful class, method, and variable names.
- Programs must compile and run without modification.
- Do not use sorting as a shortcut in this lab.

Input conventions for this version. Data arrays are non-null, contain at most 100,000 integers, and have values between $-1,000,000$ and $1,000,000$. Follow any additional range or length restrictions stated in a problem. Empty arrays are allowed where the problem permits them. Array indices start at zero. Use `long` for sums and subarray counts. Unless stated otherwise, do not modify the input arrays.

For hash-based approaches, state the assumption of average constant-time operations under suitable hashing, with amortized insertion cost. Do not present this assumption as an unconditional worst-case guarantee. Auxiliary-space analysis concerns the algorithm itself, not copies or expected-output data used only by the test driver.

Analysis Requirements

For every required problem, students must:

1. explain the main idea of the implemented approach;
2. identify the time complexity;
3. identify the auxiliary-space complexity;
4. provide appropriate test cases;
5. answer the comparison questions provided for that problem.

For selected problems, students must implement and compare multiple approaches. The purpose is to understand meaningful differences, not to invent unnecessary solutions simply to increase the number of approaches.

Testing Requirements

For each required problem, include at least three test cases:

- one normal case;
- one boundary case;
- one tricky or adversarial case where appropriate.

For each test case, record the input, expected output, and actual output.

Problem Set

Problem 1 - Count Equal-Value Blocks

Task. Count the number of maximal contiguous blocks of equal values in an integer array. A block is a consecutive group that cannot be extended left or right without including a different value. Equal values separated by other values belong to different blocks.

Example.

Input: $[4, 4, -1, -1, -1, 7, 4, 4]$

Output: 4

The blocks are $[4, 4]$, $[-1, -1, -1]$, $[7]$, and $[4, 4]$.

Required approach. Implement a single traversal that detects where a new block begins. Return zero for an empty array. A one-element array has one block. Do not store copies of the blocks or use sorting.

Analysis questions.

1. What is the input size n ? For a nonempty array, how many adjacent pairs need to be checked?
2. What are the time and auxiliary-space complexities?
3. How should the count be initialized for empty and nonempty arrays?
4. Why is counting distinct values not the same as counting blocks? Use the example to explain.
5. What results should be returned for an all-equal array and for an array whose adjacent values are always different?

Problem 2 - Suffix Maximum Queries

Task. Given an integer array and several query indices p , report the largest value from index p through index $n - 1$, inclusive. This part of the array is called a *suffix*. The array does not change between queries. Every query satisfies $0 \leq p < n$.

Example.

Array: $[3, 8, -2, 6, 1]$

Queries: $[0, 2, 4, 1]$

Output: $[8, 6, 1, 8]$

Approach A - Direct Suffix Scan. For each query, inspect all elements from the requested index through the end of the array and find their maximum.

Approach B - Suffix-Maximum Preprocessing. Traverse the array from right to left to build an additional array. At each position, store the largest value in the suffix beginning there. Answer each query using its corresponding stored value.

Implement both approaches. Process answers in query order, one at a time; a separate array of all answers is not required. An empty data array is allowed only when there are no queries. Do not modify the original array.

Analysis questions.

1. Analyze the time for one direct query in terms of both n and p . What is its auxiliary-space complexity?
2. Analyze the preprocessing time, time per query, and auxiliary space of Approach B.
3. Let q denote the number of queries. Compare the total costs, including preprocessing, when queries may start at index zero.
4. Which approach would you choose for one query at $p = n - 1$? How would your choice change for many queries on the same array?
5. How should the running maximum be initialized to handle all-negative arrays? What value is stored at the final position?
6. Why does a right-to-left traversal make the required summary available? What could become invalid if an input value changed after preprocessing?

Problem 3 - Find the Missing Number

Task. An array of length n contains n **distinct** integers from the inclusive range 0 through n . Exactly one value in that range is missing. Return the missing value. These input conditions are guaranteed; you do not need to validate them.

Example.

Input: [3, 0, 4, 1]

Output: 2

The array has length four, so the possible values are 0, 1, 2, 3, 4.

Approach A - Repeated Membership Search. Try candidate values from zero through n , in increasing order. For each candidate, search the original array from the beginning. Stop that search when a match is found. Return the first candidate that is absent.

Approach B - Presence Array. Allocate a boolean array with one position for each possible value. Mark the values found in the input, then scan the marks to find the missing value. Include allocation, initialization, marking, and the final scan in your analysis.

Implement both approaches without sorting or modifying the input. An empty array must return zero. Do not replace either required approach with an arithmetic formula, XOR method, or a shortcut based on the order of a particular test input.

Analysis questions.

1. Analyze the worst-case time and auxiliary space of repeated membership search. Give an input that forces a search for every candidate.
2. How does the work change when zero is missing instead of n ? Explain why input arrangement and the missing value can affect measurements.
3. Analyze the total time and auxiliary space of the presence-array approach.
4. Why does the presence array need $n + 1$ positions? Which input constraint makes using a value as an index safe?
5. What would need to change if values could be arbitrary negative or very large integers? Would a directly indexed array still be appropriate?
6. Why are both the distinctness and range guarantees important to the statement that exactly one value is missing?

Problem 4 - Find the First Unique Position

Task. Return the smallest index whose value occurs exactly once in the entire array. Return -1 if no such index exists. The result is an **index**, not the value at that index.

Example.

Input: `[6, 2, 6, 4, 2, 9, 4]`

Output: `5`

The value 9 occurs once and is located at index five.

Approach A - Repeated Counting. Visit candidate positions from left to right. For each candidate, scan the entire array to count occurrences of its value. Return the first index whose count is one; return -1 when all candidates have been checked without success.

Approach B - Frequency Map and Ordered Scan. Build a `HashMap<Integer, Integer>` containing the frequency of every value. Then scan the original array from left to right and return the first index with frequency one.

Implement both approaches. An empty array returns -1 ; a one-element array returns zero. A value seen once so far is not necessarily unique in the complete input.

Analysis questions.

1. Analyze the worst-case time and auxiliary space of repeated counting. Give an input on which every candidate must be examined.
2. Analyze the average total time of the frequency-map approach, including both passes. Let d be the number of distinct values when describing storage.
3. Why is a `HashSet` containing only distinct values insufficient for the required frequency check?

4. Why must the second pass follow the original array order rather than return an arbitrary value with frequency one?
5. How much work does each required approach perform when every array value is distinct? Explain any early return.
6. How does returning an index avoid confusing the legitimate data value -1 with the no-result marker?

Problem 5 - Sum of All Subarray Sums

Task. Calculate the sum of the sums of **all nonempty contiguous subarrays**. Each distinct pair of start and end indices defines one subarray. Include every such subarray exactly once, even when several subarrays contain the same values.

Example.

Input: $[2, -1, 3]$

Output: 11

The six subarray sums are 2, -1 , 3, 1, 2, and 4. Their total is 11.

Approach A - Nested-Loop Accumulation. Fix a starting index, extend the ending index one position at a time, and maintain the current subarray sum. Add that sum to the final total after every extension. Repeat for every starting index. Do not use a third loop to recompute each sum.

Approach B - Element Contributions. For each position, determine how many subarrays include it by counting the possible starting and ending positions. Multiply its value by this number and add its contribution to the total, without enumerating all subarrays.

Implement both approaches. **For this problem only**, $0 \leq n \leq 10,000$; the general value limits still apply. Use `long` for subarray sums, contributions, and the result. Promote an operand to `long` *before* multiplication, not only when storing the result. An empty array returns zero.

Analysis questions.

1. How many nonempty subarrays exist? Use this count to analyze the time of Approach A and state its auxiliary space.
2. For an index i , how many starting positions and ending positions can form a subarray containing $a[i]$? Derive its contribution.
3. Analyze the time and auxiliary space of Approach B. Why can it avoid generating the subarrays explicitly?
4. Why does this method remain correct when values are negative or zero? What should a one-element array return?
5. Why may an `int` calculation overflow even when the final variable is a `long`? Explain why the stated size and value limits keep the total within `long` range.

Problem 6 - Interleave Two Array Halves

Task. An array has even length $n = 2m$. Interleave its two halves while preserving the order within each half. Both approaches must modify the original array to contain the result.

Example.

Input: [1, 2, 3, 10, 20, 30]

Output: [1, 10, 2, 20, 3, 30]

In general, $[x_1, \dots, x_m, y_1, \dots, y_m]$ becomes $[x_1, y_1, \dots, x_m, y_m]$.

Approach A - Auxiliary Array. For $n \geq 4$, allocate another array and fill it by alternately taking the next value from each original half. Copy the complete result back to the input array. Include allocation and both passes in your analysis.

Approach B - In-Place Ordered Insertions. For $n \geq 4$, repeatedly take the next unplaced value from the second half and insert it immediately after the next unplaced value from the first half. Save the selected value in a temporary variable and shift the intervening values one position to the right. Maintain indices as the positions change; do not allocate another array or collection proportional to n .

Implement both approaches. Lengths zero and two are already correctly arranged; return without allocating an extra array. Odd lengths are outside this problem's input conditions. Test each method on a fresh copy of the same input. This is a positional rearrangement, not sorting.

Analysis questions.

1. Analyze the time and auxiliary space of the auxiliary-array approach, including copying back.
2. Trace the ordered insertions on the example. How many values must be shifted for the first insertion, the next insertion, and so on? Use $m = n/2$ to derive the total shifting work.
3. Analyze the time and auxiliary space of the required in-place approach. Explain the trade-off compared with Approach A.
4. Why must the selected value be saved before shifting? Why does the shift proceed from right to left?
5. How does your solution preserve the relative order within both halves, including when values repeat?
6. Does using less auxiliary memory necessarily mean using less time? Base your answer on these two required algorithms, not a claim about every possible in-place algorithm.

Challenge Problems

The challenge problems are optional. Complete the required problem set first.

Challenge 1 - Count Subarrays with a Target Sum

Task. Given an integer array and a target T , count the nonempty contiguous subarrays whose sum equals T . Different start/end index pairs count separately, even when they contain equal values. Use the general array limits and $-10^{11} \leq T \leq 10^{11}$.

Example.

Array: $[1, -1, 1, 0]$

$T = 1$

Output: 5

The qualifying index pairs are $(0, 0)$, $(0, 2)$, $(0, 3)$, $(2, 2)$, and $(2, 3)$.

Develop at least two approaches without sorting. Compare nested-loop accumulation with a method that tracks the frequencies of previously observed prefix sums in a HashMap. Use `long` for the target, prefix sums, and returned count. An empty array returns zero.

For each approach, analyze time and auxiliary space. Explain why prefix-sum *frequencies* are needed rather than only membership or one stored index. Account for subarrays starting at index zero, and explain why the current prefix must be queried before its frequency is increased. Include negative values and zeros; $[0, 0, 0]$ with $T = 0$ must return six, without counting empty subarrays.

Challenge 2 - Longest Subarray with Distinct Values

Task. Return the length of the longest contiguous subarray in which no value occurs more than once. Elements cannot be skipped. Return zero for an empty array.

Example.

Input: $[4, 2, 4, 5, 2, 3]$

Output: 4

The subarray $[4, 5, 2, 3]$ has four distinct values.

Develop at least two approaches without sorting. First, start at each position and extend a candidate subarray, using a fresh HashSet to stop at its first repeated value. Then investigate a moving-window approach that stores each value's most recent index in a HashMap and adjusts the left boundary when necessary.

Analyze time and auxiliary space under the stated hashing assumption, including repeated set construction. Explain why the efficient window's left boundary must never move backward. Trace $[1, 2, 2, 1]$, whose answer is two, and explain how an occurrence before the current window differs from one inside it. Include all-distinct and all-equal arrays.

Runtime Experiment

Use **Problem 3 - Find the Missing Number** for a small empirical experiment.

Implement repeated membership search and the presence-array approach. Measure their approximate execution time for increasingly large input arrays.

Suggested input sizes:

1,000

5,000

10,000

50,000

Input construction. Use $a[i] = i$ for $0 \leq i < n$. These inputs contain all values from zero through $n - 1$, so the missing number is n . Generate the array before timing. Implement the required algorithms rather than using a shortcut based on this particular construction.

Use `System.nanoTime()` or another instructor-approved timing mechanism.

Measurement procedure.

1. Run both methods several times on a small input before collecting measurements.
2. Exclude input generation, printing, and correctness checks from timing. Include fresh presence-array allocation, initialization, marking, and the final scan in every timed run of Approach B.
3. Repeat each feasible measurement at least three times. Verify the returned result after every run and record the median time in milliseconds.
4. Use the same computer and array for both methods. Record the Java version, hardware, and number of runs.
5. If a large run is impractical, mark it **not run** and explain why. Never invent or extrapolate a measured time. Add smaller sizes if needed to obtain at least three paired measurements.

Record your results in a table similar to the following. The expected answer is n in every row.

n	Repeated Search (ms)	Presence Array (ms)
1,000		
5,000		
10,000		
50,000		

Reflection. Do the measurements generally agree with your complexity analysis? Explain any irregularities briefly. Why does making n the missing value prevent an early return after only one candidate search? Predict how the comparison would change for $a[i] = i + 1$, where zero is missing; additional measurements for this alternative are optional. Exact timings are not expected to be identical across computers.

Algorithm Analysis Template

For each problem requiring multiple approaches, summarize the comparison using a table such as the following.

Criterion	Approach A	Approach B
Main idea		
Time complexity		
Auxiliary-space complexity		
Advantages		
Disadvantages		
Suitable when		

Conclude each comparison with a short answer to:

Which approach would you choose for the stated constraints, and why?

Report Requirements

Submit one PDF report. Identify your full name, student ID, course, and Lab 1 – Version 4 on the first page. The report does not need to repeat the complete problem statements.

For each required problem, include:

1. a short explanation of the implemented algorithm(s);
2. time-complexity analysis;
3. auxiliary-space analysis;
4. answers to the problem-specific analysis questions;
5. the required test cases with expected and actual outputs.

For Problems 2–6, include the Algorithm Analysis Table comparing the required approaches.

Include the runtime experiment and reflection. For Problem 2, separate preprocessing, per-query, and total costs. Put completed challenges in separate sections.

Submission Requirements

Each student must submit one compressed ZIP file containing:

- all Java source-code files;
- one PDF report;

Recommended structure:

```
StudentID_FullName_Lab01_V4.zip
|
|-- src/
| |-- Problem01_EqualValueBlocks/
| |-- Problem02_SuffixMaximumQueries/
| |-- Problem03_MissingNumber/
| |-- Problem04_FirstUniquePosition/
| |-- Problem05_AllSubarraySums/
| |-- Problem06_InterleaveHalves/
| |-- Challenge01_TargetSumCount/ (if completed)
| |-- Challenge02_DistinctSubarray/ (if completed)
|
|-- report/
| |-- Lab01_V4_Report.pdf
```

Include the runtime-experiment driver in `src/` with brief compile/run instructions. Submit actual `.java` files; screenshots or code pasted into the report do not replace the source files. Do not include compiled `.class` files or IDE build folders.

Final Checklist

Before submitting, make sure that:

- all required Java programs compile and run;
- the report includes time and auxiliary-space analysis;
- multiple approaches are implemented where required;
- comparisons explain meaningful trade-offs rather than only listing Big-O values;
- test cases include normal, boundary, and tricky situations;
- the runtime experiment is included;
- the ZIP file follows the requested organization and naming convention.

The objective of this lab is not only to solve each problem, but to understand why different algorithms behave differently and when one approach should be preferred over another.